TissueGraph

A blueprint for differentiable tissue: hierarchical sets, operators, and an LLM-driven simulation layer

Janelia Research Campus

June 15, 2026

Abstract

Living tissues couple biochemical signalling, cellular interaction, mechanical force, and collective dynamics across scales. The frameworks built so far (CellGraph, NeuralGraph, MPMGraph, MetabolismGraph) each implement *one* graph type; a tissue needs them coexisting. This document is a single blueprint, distilled from a working prototype, for building one repository that unifies them. It has three parts: (i) the missing primitive — a **hierarchical graph container** stated in the language of **sets** and **operators**; (ii) the **contract** that lets a large language model (LLM) drive a registry of low-level operators through a declarative *simulation* specification; and (iii) the **experiment** — ten distinct simulations and an emergent ant-trail model, all produced from the same code by changing only the simulation file — with the lessons that follow for how to construct the repository.

Part I — The abstraction

1 Two entities: sets and fields

The container is built from two kinds of object: **sets** and **fields**. A **set** S_k is a discrete collection of like entities at scale k — metabolites, particles, cells, populations, organisms (Fig. 1). Adjacent scales are linked by **parent maps**

$$\pi_k : S_k \longrightarrow S_{k+1},$$

where $\pi_k(x)$ is the higher-level object that contains x. A cell is built from two kinds of S_0 entity — its **particles** (mechanics) and its **metabolites** (chemistry) — and π_0 sends each of them to the cell it belongs to, so $\pi_0(x) = c$ reads "x belongs to cell c". The constituents of a given cell c are its *fibre*, the set of everything that maps to c,

$$\pi_0^{-1}(c) = \{\, x \in S_0 : \pi_0(x) = c \,\},$$

so a cell simply *is* the collection of its particles and metabolites. A **field** $f : \Omega \to \mathbb{R}^q$ is a continuous quantity defined on its own grid — morphogen concentration, pressure, substrate stiffness, and so on. Fields are *not* part of the containment hierarchy: they neither contain nor are contained by cells. Instead, each field is bound to exactly one set level through an *exchange* relation (particle–grid transfer, cell sampling, or population-level secretion and sensing); these couplings are the operators of Section 2.

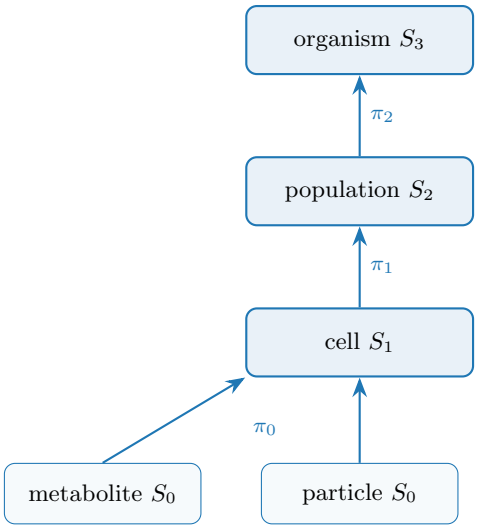

Figure 1: The containment hierarchy: the parent map π_k sends each entity to the object that contains it. A cell is the fibre $\pi_0^{-1}(c)$ — its particles and metabolites.

2 Four operators

A GNN is an **operator** $\mathcal{O}$ returning a time-derivative contribution, dispatched by the relation it acts on. We use four kinds, one per relation the container admits (Fig. 2):

Lateral	$\mathcal{O}_E,\ E \subseteq S_k \times S_k$	(ODE interaction + graph Laplacian)
Aggregate $\uparrow$	$\sum_\pi : S_k \to S_{k+1}$	(reduction over fibres of π)
Broadcast $\downarrow$	$\pi^* : S_{k+1} \to S_k$	(lift along π)
Exchange	$\mathcal{S}, \mathcal{G} : S_k \leftrightarrow F$	(scatter / gather; bipartite)

Two pieces of notation: $S_k \times S_k$ is the set of all node *pairs*, so an edge set $E \subseteq S_k \times S_k$ is simply a graph on S_k — which nodes interact (e.g. neighbours within a cutoff radius, or a fixed connectome). And $\mathcal{S}, \mathcal{G}$ are the two halves of a field coupling: *scatter* $\mathcal{S} : S_k \to F$ writes object state into the field, while *gather* $\mathcal{G} : F \to S_k$ reads it back. Each operator in turn:

Lateral $\mathcal{O}_E$. Message passing *within* one scale along the edges E. It carries the within-scale physics: pairwise interactions (forces, adhesion, signalling) plus a graph Laplacian that diffuses or smooths quantities over the edges. Examples: particle–particle elasticity, cell–cell flocking or adhesion, neuron–neuron signal propagation.

Aggregate $\sum_\pi$ (up). Pools each parent's fibre into one quantity — a cell's position is the mean of its particles, its metabolic state the sum over its molecules. This is how fine-scale dynamics are summarised into, and so move, the coarser object above.

Broadcast π^* (down). The adjoint of Aggregate: it copies a parent's state to every child in its fibre, so a decision taken at the cell level (a body force, a polarity, a signalling output) is applied identically to all of that cell's particles. Aggregate and Broadcast are a matched up/down pair on the same map π.

Exchange $\mathcal{S}, \mathcal{G}$. Couples a set to a field: scatter $\mathcal{S}$ deposits/secretes object state into the field, gather $\mathcal{G}$ samples/senses the field back onto the objects. The relation is bipartite (objects $\leftrightarrow$ grid); it is exactly the MPM particle–grid transfer (P2G/G2P) and chemotactic secretion/sensing.

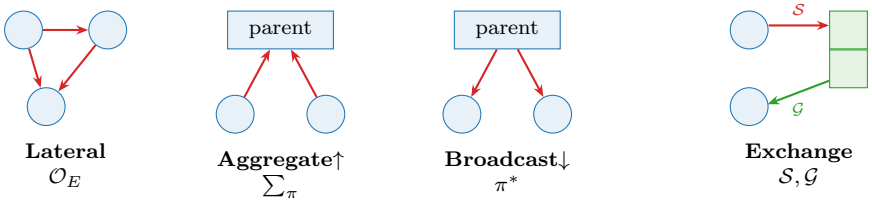

Figure 2: The four operators. *Lateral* acts within a set; *Aggregate* and *Broadcast* move across containment π; *Exchange* scatters ($\mathcal{S}$) to and gathers ($\mathcal{G}$) from a field. P2G/G2P (mechanics) and the metabolite–reaction graph are both Exchange.

3 Dynamics: a schedule

A simulation is a multi-rate composition of operators, declared as a `Schedule`. In math terms, one tick is the composition $\Phi = \mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$, and the dynamics is its iteration $x_{t+1} = \Phi(x_t)$ — a discrete flow (with "multi-rate" meaning some operators fire only on every k-th tick). The `Schedule` is just the code-level encoding of that composition. Each operator is *pure*: instead of editing the state in place, it only *returns its contribution* — a delta Δ_i (a time-derivative term) for the sets it touches. The schedule sums the deltas acting on each set and integrates once per tick, $x_{t+1} = x_t + \Delta t \sum_i \Delta_i$. Two consequences follow: several operators may act on the same set in one tick (their deltas simply add — none overwrites another), and because nothing is mutated in place, gradients flow through the sums and the integrator, so the whole rollout stays differentiable. The LLM searches over schedules and operator parameterizations — one well-defined action space instead of per-domain code.

4 Glossary

Concept	Set/operator term	Symbol	Code
collection of like nodes	set (level)	S_k	`Level`
a single node	element	$x \in S_k$	row of `Level.state`
learnable per-node code	embedding	a_i	`Level.embedding`
containment	partition	$\pi_k : S_k \to S_{k+1}$	`Level.parent`
within-set links	relation	$E \subseteq S_k \times S_k$	`Level.edge_index`
continuum quantity	field	$f : \Omega \to \mathbb{R}^c$	`Field`
dynamics (GNN)	operator: ODE+Laplacian	$\mathcal{O}$	`Operator`
within-set op	lateral	$\mathcal{O}_E$	`Lateral`
up op	reduction over fibres	$\sum_\pi$	`Aggregate`
down op	lift	π^*	`Broadcast`
set↔field op	transfer	$\mathcal{S}, \mathcal{G}$	`Exchange`
the container	hierarchy	$(S_\bullet, F_\bullet, \pi_\bullet)$	`Hierarchy`
the model	composition of ops	$\mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$	`Schedule`

Part II — The LLM ↔ registry methodology

5 Two layers and the contract between them

The prototype's purpose was not the tissue physics but figuring out *how an LLM should drive a registry of low-level operators*. The answer is a strict two-layer split that meet only at a declarative, validated file — the **simulation** specification.

INTENT	natural language ("2000 cells, two mechanical types, a chemical that guides only one")
↓ *LLM compiles*	bounded, schema-checked generation
SIMULATION	`simulation.yaml` — sets · fields · operators(+selectors) · schedule
↓ *engine resolves*	names looked up in the registry
REGISTRY	`@register_operator` / `@register_field` — the only place code lives
↓	
ENGINE	generic: build `Hierarchy` → run `Schedule` → `zarr` (written once)

The **simulation file is the contract**. The LLM owns everything above it (intent → simulation); the code owns everything below (operators). They never touch directly. This makes the system both LLM-drivable and human-auditable: the simulation is small, typed, and reviewable, unlike the 500-line flat `config.py` it replaces. Three properties make the contract enforceable, and they answer the three questions this experiment was built around (Table 1).

Question	Mechanism	Where
archive simulations	spec + seed + metrics + thumbnail, name-keyed	`archive.py`, `GALLERY.md`
missing model	registry lookup errors with the available set; add one `Operator`	`registry.get_operator`
missing property	operators *declare* what they require; validator checks	`Operator.REQUIRES_*`

Table 1: The three operational modes.

6 The simulation specification

A simulation is built from one vocabulary: **sets** (a leaf set declares its `parent` and `per_parent`), **fields** (each `couples_to` one set), **operators** (each acting at a *selector* `set` or `set[attr=value]`), and a **schedule** (the per-frame ordered list). Listing 1 is a complete example.

Listing 1: A complete simulation: soft cells secrete and follow a chemical and self-aggregate.

```
name: aggregate
seed: 0
n_frames: 250
sets:
  cell:
    n: 100
    types: {soft: {fraction: 0.5, youngs: 12}, stiff: {fraction: 0.5, youngs: 300}}
  particle: {parent: cell, per_parent: 100, radius: 0.02}   # containment
fields:
  chemical: {frame: grid, res: 96, diffusion: 0.1, decay: 0.05, couples_to: cell}
operators:
  - {op: motility, at: cell, speed: 15.0, rot: 0.4}
  - {op: mpm, at: particle, n_grid: 128, substeps: 8, a_max: 800, drag: 4}
  - {op: secrete, at: "cell[type=soft]", to: chemical, rate: 1.0}    # only pop 1 makes it
  - {op: sense,  at: "cell[type=soft]", from: chemical, gain: 150.0} # only pop 1 follows it
schedule: [aggregate, [motility], secrete, chemical.diffuse, sense, mpm]
```

The validator (prototype: `scenario_schema.py`) guarantees that a file which loads is runnable: every `op` exists; every `at`/`to`/`from` references a declared set/field; type `fraction`s sum to one; every schedule token resolves. (One trap: never use a YAML 1.1 boolean key such as `on`/`off` — we use `at`.)

7 Operator catalog

Operators are the system's vocabulary; it grows monotonically and every past simulation keeps working. The prototype catalog:

op	kind	acts on	behaviour
`boids`	lateral	cell	collective motion (cohesion/align/separate)
`motility`	lateral	cell	active self-propulsion (wandering heading)
`adhesion`	lateral	cell	type-specific cohesion → sorting
`mpm`	exchange	particle	MLS-MPM mechanics; soft/stiff via `youngs`
`secrete`	exchange	cell→field	deposit into a field
`sense`	exchange	cell←field	chemotactic accel up-gradient
`trail`	exchange	cell	Physarum 3-sensor pheromone steering (ants)
`aggregate`	builtin	—	cell pos/vel = mean of its particles (↑)
`<field>.diffuse`	builtin	—	field self-update (Laplacian + decay)

8 Natural language → simulation: the repeatable mapping

To make the LLM's compilation *repeatable* (same request → same structure), it follows fixed rules; the most useful:

1. "N cells/agents" → a set with `n: N`.
2. "made of K X per cell" → a contained set with `parent`/`per_parent` (two levels).
3. "two types differing by P" → `types:` with P as a per-type property.
4. "mechanical/elastic/rigid" → particles + `mpm`; map stiffness to `youngs`.
5. "moves by boids / flock" → `boids`; "wanders / motile" → `motility`.
6. "creates/secretes a chemical" → a `field` + `secrete`; "follows / chemotaxis" → `sense`; "ant trails" → `trail`.
7. **"only population X does Y"** → the selector `at: cell[type=X]`. This is the highest-leverage primitive: it scopes behaviour to a subpopulation and lets two behaviours coexist.

9 When the present code is not enough (a missing model)

The pipeline fails loudly and points at the gap, e.g. for a request needing field advection or reaction–diffusion:

```
operator 'reaction_diffusion' not in registry.
Available: ['adhesion','boids','mechanics','motility','mpm','secrete','sense','trail']
```

The fix is **one new `Operator` subclass** — never an engine change: write `forward(self, H, mask) -> set: accel` (or mutate a field, return {}); decorate `@register_operator`; declare its needs; reference it in the simulation. We exercised this live twice during the experiment: adding `adhesion` unlocked cell *sorting*, and adding `trail` unlocked ant *trail-following* — each ∼20 lines, with the engine and all other simulations untouched.

10 When a cell/particle property does not exist

Operators *declare* what they need (`REQUIRES_PARAMS`, `REQUIRES_TYPE_PROPS`); the validator checks before running, resolving a property along the containment chain (`mpm` acts on particles but `youngs`

lives on the parent cell's types):

```
# missing required property -> hard error, before any computation:
operator 'mpm' requires property 'youngs' on every type of 'cell'; missing on type 'soft'.
# property no operator reads -> warning (typo guard), run proceeds:
[warn] property 'viscosity' on cell.soft is read by no operator (known: ['fraction','youngs'])
```

Design rule: capabilities are declared, not discovered at a crash. The operator that consumes a property is the single place that declares the requirement, turning a would-be `AttributeError` thousands of substeps deep into a one-line message up front.

Part III — The experiment

11 A spread of simulations from one registry

Once the vocabulary existed, ten qualitatively different behaviours were *just different YAML* over the same code (Fig. 3). Breadth comes from the intermediate representation, not from new code. Each is 600 cells or 100 cells of ∼100 MLS-MPM particles; soft cells are red, stiff blue, the chemical field green.

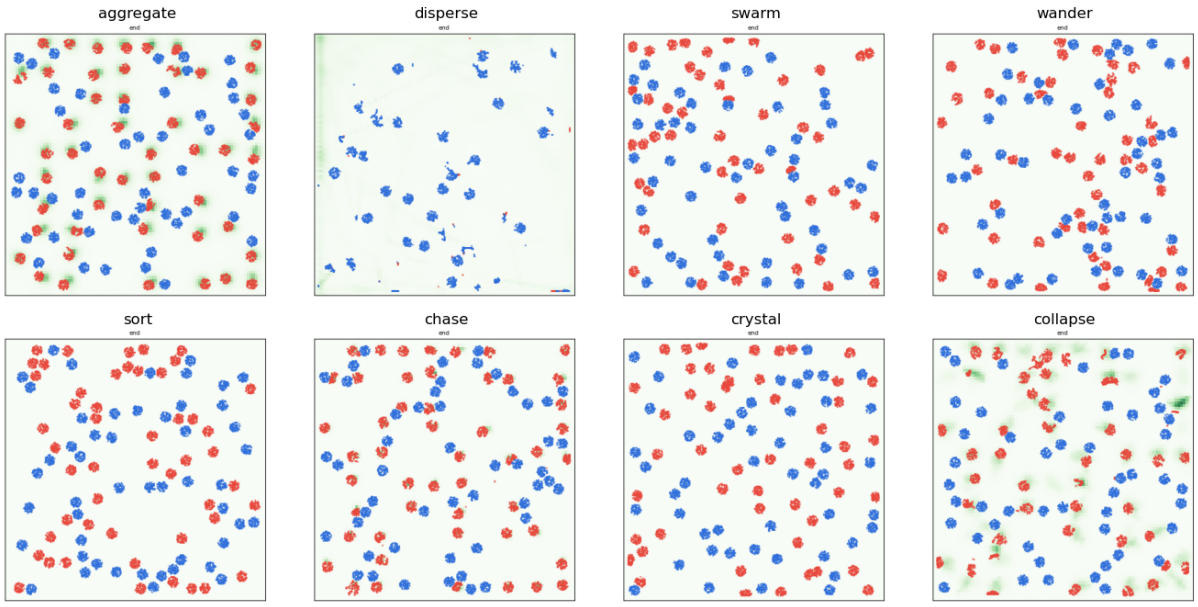

Figure 3: End states of eight simulations, all generated from the same operator registry by changing only the simulation file. *aggregate*: soft cells cluster on self-secreted chemical; *disperse*: soft flee their own chemical; *swarm*: boids flock; *wander*: active gas; *sort*: type-specific adhesion segregates the populations; *chase*: a fast-decaying field makes moving hotspots; *crystal*: pure repulsion spaces cells evenly; *collapse*: low diffusion + strong chemotaxis → one tight cluster.

12 Emergent ant trails

Adding the `trail` operator (a Physarum/Jones three-sensor rule: each ant samples pheromone ahead-left / ahead / ahead-right and turns toward the strongest, while `motility` drives it forward

and `secrete` lays pheromone) yields self-reinforcing trails that grow, are followed, and coarsen over long runs (Fig. 4). Listing 2 is the full simulation.

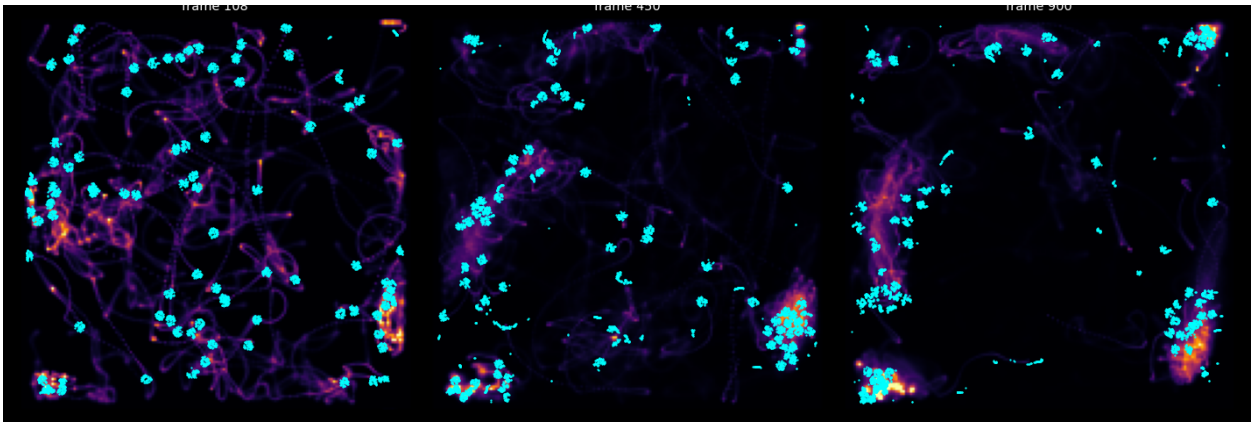

Figure 4: Ant trail-following over a long run (pheromone in inferno, ant cells in cyan). A diffuse network of trails forms (left), reinforces, and *coarsens into a few dominant trails* with ants marching along them (right) — classic stigmergy.

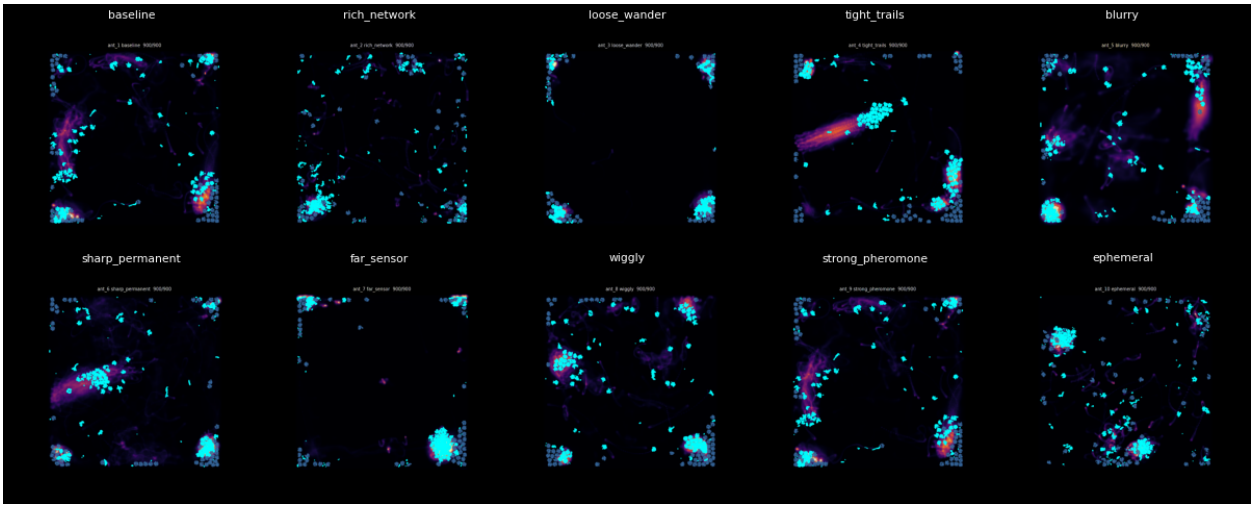

Figure 5: Ten ant variants, each the same `trail+motility+secrete` code with a few parameters changed (turn rate, sensor reach/angle, pheromone diffusion/decay, deposit rate, rotational noise). The parameter axes span the trail-formation regime — from loose wandering to tight reinforced networks.

Listing 2: The ant simulation: red cells lay, follow, and reinforce pheromone trails.

```
name: ant
seed: 0
n_frames: 1500
sets:
  cell:
    n: 150
    types: {soft: {fraction: 0.6, youngs: 60}, stiff: {fraction: 0.4, youngs: 300}}
  particle: {parent: cell, per_parent: 60, radius: 0.012}
fields:
```

```
  pheromone: {frame: grid, res: 160, diffusion: 0.02, decay: 0.04, couples_to: cell}
operators:
  - {op: motility, at: cell, speed: 300.0, rot: 0.06}            # persistent forward motion
  - {op: trail,  at: "cell[type=soft]", from: pheromone, turn: 0.4, sensor_dist: 0.04, sensor_angle: 0.5}
  - {op: secrete, at: "cell[type=soft]", to: pheromone, rate: 2.0}
  - {op: mpm, at: particle, n_grid: 128, substeps: 8, a_max: 1200, drag: 0.6}
schedule: [aggregate, trail, motility, secrete, pheromone.diffuse, mpm]
```

13 What the process taught us

1. **The intermediate representation is the whole game.** Ten distinct behaviours and an ant model were all different YAML over the same code.
2. **Selectors are the highest-leverage primitive.** `cell[type=soft]` expresses "only population 1 does X" and lets behaviours coexist; every operator must honour the selector mask.
3. **Verification must check *intent*, not just "it ran".** The hard bugs were physical: unstable diffusion ($dt \cdot D > 0.25$), wrong particle volume, unclamped accelerations blowing up MPM, cell inversion at very low stiffness, GPU non-determinism breaking reproducibility, and — subtlest — the *wrong metric* (net displacement vs. nearest-neighbour clustering).
4. **Honesty about regime.** Self-secreted chemotaxis aggregates only below a density threshold (above it the field is spatially flat — correct physics); the simulation/verify output should state the regime, not silently show the one setting that worked.
5. **Determinism is a feature you must force** (deterministic CUDA kernels), or "reproduce with a new seed" is meaningless.

Part IV — Building the repository

14 Proposed methodology

Layering (strict; no leaks across).

- `models/base.py` — entities, operator base, the capability-contract fields.
- `models/registry.py` — the only name→code map (entity / operator / field).
- `models/<domain>/*.py` — operators, one concern per file, each declaring `REQUIRES_*`.
- `simulation.py` — the schema + validator (the contract gatekeeper).
- `engine.py` — generic; contains **no** simulation-specific logic.
- `viz.py` — generic over the `Hierarchy/zarr`; swappable backends (2-D now, 3-D later).
- `archive.py` + `simulations/` — the growing, reproducible gallery.

The build loop (how LLM and code co-evolve).

1. user states intent in natural language;
2. LLM compiles → `simulation.yaml` (following the mapping rules);

3. validator runs: unknown op → write one operator; missing property → fix spec or add the requirement; else it is runnable;
4. engine runs; **verify** checks intent + repeatability + well-formedness;
5. `archive.py` records spec+seed+metrics+thumb into the gallery;
6. new capability → one new registered operator (+ its `REQUIRES_*`), never an engine edit.

Invariants to hold the line.

- Code lives only behind the registry; the engine never branches on simulation names.
- Every operator honours its selector mask and declares its `REQUIRES_*`.
- Every simulation ships an intent check; "it ran" $\neq$ "it worked".
- Archive the recipe (spec+seed); regenerate the dish.
- When a result holds only in a regime, the spec/verify says so.

This is the contract that lets the LLM operate at the top (intent → simulation, plus occasional single-operator authoring) while the low-level code stays a clean, growing, verifiable registry underneath.